AKDE Current Math Formulas

Generated from the active TypeScript implementation

1 Scope

This document lists the formulas currently used by the AKDE implementation under `kirigami/model/`. It reflects the active code paths, including clamps, validation gates, and the currently selected minor-cut formula.

2 Inputs and Symbols

$$
\begin{aligned}
N &:= \text{edge count}, \qquad N \geq 3,\\
L &:= \text{base edge length (mm)},\\
L_o &:= \text{outer polygon-face edge length on the perimeter (mm)},\\
H &:= K_{\text{tot}} = \text{vertical apex height (mm)},\\
T &:= \text{material thickness (mm)}.
\end{aligned}
$$

The implementation validates

$$H > 0, \qquad T > 0.$$

References: Implementation authority: `kirigami/model/validation.ts`.

3 Derived Pyramid Geometry

3.1 Primary lengths and angles

$$R = \frac{L}{2\sin\left(\frac{\pi}{N}\right)}, \tag{1}$$

$$s = \sqrt{R^2 + H^2}, \tag{2}$$

$$\psi = \operatorname{atan2}(H, R), \tag{3}$$

$$\kappa = \frac{H}{R}. \tag{4}$$

References: For the regular-N-gon circumradius relation and non-right-triangle trigonometry, see OpenStax, "10.1 Non-right Triangles: Law of Sines" and "7.2 Right Triangle Trigonometry."

The apex face angle is

$$\eta = \begin{cases} 2\arcsin\left(\min\left(1, \dfrac{R\sin(\pi/N)}{s}\right)\right), & s > 0 \text{ and } R > 0,\\ 0, & \text{otherwise.} \end{cases} \tag{5}$$

References: The η expression is the law-of-sines reduction for the lateral face isosceles triangle; see OpenStax, "10.1 Non-right Triangles: Law of Sines." AKDE-specific specialization: `kirigami/model/geometry.ts`.

The discrete angle defect and uniform molecule angle are

$$\delta_{\text{apex}} = 2\pi - N\eta, \tag{6}$$

$$\theta = \frac{\delta_{\text{apex}}}{N}, \tag{7}$$

$$\tau = \frac{2\pi}{N}. \tag{8}$$

References: For angle defect as 2π minus the sum of incident angles, see Keenan Crane, *Discrete Differential Geometry: An Applied Introduction*, §5.5–5.6, especially the angle-defect definition and discrete Gauss–Bonnet discussion: CMU notes PDF. For the kirigami reduction $\theta = \delta_{\text{apex}}/N$ and closure step $\tau = 2\pi/N$, see Liu et al., "Programmable Kirigami," Eq. (1) and Figure 2.

The molecule width is

$$w = 2s \sin\left(\frac{\theta}{2}\right). \tag{9}$$

References: The paper defines $W(i, j)$ as the edge-molecule width and Figure 2 gives the symmetric trapezoid geometry; the chord specialization used here is an AKDE uniform-fan derivation built on Liu et al., "Programmable Kirigami," Figure 2 and Eq. (2). Implementation authority: `kirigami/model/geometry.ts`.

3.2 Major cut and molecule measures

The physical major-cut radius is

$$r_{\text{phys}} = \frac{T}{\sin(\theta/2)}.$$

The implementation applies a visualization clamp:

$$r_{\text{apex}} = \begin{cases} \min\left(\dfrac{T}{\sin(\theta/2)},\, 0.4\,s\right), & T > 0 \text{ and } \theta > 0, \\ 0, & \text{otherwise.} \end{cases} \tag{10}$$

The molecule end-leg length is

$$D = 2s \tan\left(\frac{\theta}{2}\right) = \frac{w}{\cos(\theta/2)}, \qquad 0 < \theta < \pi. \tag{11}$$

References: Major/minor cuts and molecule geometry are introduced in Liu et al., "Programmable Kirigami," Figure 2 and the paragraph introducing major and minor cuts. The explicit formulas for r_{apex}, the $0.4s$ clamp, and D are AKDE implementation formulas in `kirigami/model/geometry.ts`.

3.3 Dihedral angle

Let

$$\phi = -\frac{\pi}{2}, \qquad \Delta = \frac{2\pi}{N}, \qquad \mathbf{a} = (0, 0, H),$$

and define base vertices

$$\mathbf{b}(\alpha) = \big(R\cos\alpha,\ R\sin\alpha,\ 0\big).$$

Then

$$\mathbf{b}_i = \mathbf{b}(\phi),$$
$$\mathbf{b}_{\text{prev}} = \mathbf{b}(\phi - \Delta),$$
$$\mathbf{b}_{\text{next}} = \mathbf{b}(\phi + \Delta).$$

The adjacent face normals used by the code are

$$\mathbf{n}_1 = (\mathbf{b}_{\text{prev}} - \mathbf{b}_i) \times (\mathbf{a} - \mathbf{b}_i), \tag{12}$$

$$\mathbf{n}_2 = (\mathbf{a} - \mathbf{b}_i) \times (\mathbf{b}_{\text{next}} - \mathbf{b}_i). \tag{13}$$

If either normal has near-zero length, the implementation returns $\gamma = \pi$. Otherwise,

$$\alpha_{\text{norm}} = \arccos\left(\text{clamp}_{[-1,1]} \frac{\mathbf{n}_1 \cdot \mathbf{n}_2}{\|\mathbf{n}_1\|\|\mathbf{n}_2\|}\right), \tag{14}$$

$$\gamma = \pi - \alpha_{\text{norm}}. \tag{15}$$

References: For cross products and normals to the plane spanned by two vectors, see OpenStax, "2.4 The Cross Product." The specific adjacent-face construction and $\gamma = \pi - \alpha_{\text{norm}}$ convention are AKDE implementation choices in `kirigami/model/geometry.ts`.

4 Minor-Cut Formulas Currently Active

The active implementation selects `MINOR_CUT_FORMULA = "fold-reach"`.

First, the fold-clearance depth used by constraint $C6$ is

$$d_{\text{clear}} = \begin{cases} T\tan(\gamma/2), & T > 0,\ 0 < \gamma/2 < \pi/2, \\ 0, & \text{otherwise.} \end{cases} \tag{16}$$

The active minor-cut length is

$$\ell = \sqrt{\left(\frac{w}{2}\right)^2 + r_{\text{apex}}^2}. \tag{17}$$

This is the same as

$$\ell = \text{hypot}\left(\frac{w}{2},\, r_{\text{apex}}\right).$$

References: Liu et al. state that the minor cut depends on "the angle between adjacent mesh faces and the width of the hinge" and refer to Figure 2, but they do not give a closed-form equation; see "Programmable Kirigami," the paragraph immediately before Figure 2. The formulas $d_{\text{clear}} = T\tan(\gamma/2)$ and $\ell = \sqrt{(w/2)^2 + r_{\text{apex}}^2}$ are AKDE-specific constructions in `kirigami/model/geometry.ts`; the current branch is `MINOR_CUT_FORMULA = "fold-reach"`.

5 Constraint Equations

The code uses a tolerance

$$\varepsilon = 10^{-10}.$$

5.1 $C1$: angle closure

$$\rho_{C1} = |N\theta + N\eta - 2\pi|, \tag{18}$$

$$C1 \text{ is satisfied iff } \rho_{C1} < \varepsilon. \tag{19}$$

References: Directly from Liu et al., Eq. (1): "Programmable Kirigami."

5.2 $C2$: vector closure

The phase sequence is

$$\Phi_n = n\tau, \qquad n = 0, 1, \ldots, N-1.$$

The closure sum is

$$S_x = \sum_{n=0}^{N-1} w\cos(\Phi_n), \tag{20}$$

$$S_y = \sum_{n=0}^{N-1} w\sin(\Phi_n), \tag{21}$$

with residual

$$\rho_{C2} = \sqrt{S_x^2 + S_y^2}, \tag{22}$$

$$C2 \text{ is satisfied iff } \rho_{C2} < \varepsilon. \tag{23}$$

References: Directly from Liu et al., Eq. (2) and the closed-polygon condition explained immediately below it in "Programmable Kirigami."

5.3 $C3$: molecule angle range

$$\rho_{C3} = \max(0,\ |\theta| - (\pi - \varepsilon))\,, \tag{24}$$

$$C3 \text{ is satisfied iff } |\theta| < \pi - \varepsilon. \tag{25}$$

References: Directly from Liu et al., inequality (3) in "Programmable Kirigami."

5.4 $C4$: non-negative width

$$\rho_{C4} = \begin{cases} -w, & w < 0, \\ 0, & w \geq 0, \end{cases} \tag{26}$$

$$C4 \text{ is satisfied iff } w \geq -\varepsilon. \tag{27}$$

References: Directly from Liu et al., inequality (4) in "Programmable Kirigami."

5.5 $C5$: fold overlap

$$\rho_{C5} = \max(0,\ w - L), \tag{28}$$

$$C5 \text{ is satisfied iff } \rho_{C5} \leq \varepsilon, \tag{29}$$

equivalently

$$w \leq L.$$

References: This is an AKDE validity proxy, not a closed-form condition from the paper. The paper states that the molecule must be valid without overlapping; AKDE instantiates that idea as $w \leq L$. Implementation authority: `kirigami/model/constraints.ts`; conceptual context from Liu et al., the text introducing inequalities (3)–(4).

5.6 $C6$: cut vs. dihedral

The code computes

$$d_{\text{relief}} = \begin{cases} T\tan(\gamma/2), & 0 < \gamma/2 < \pi/2 - 10^{-12}, \\ \infty, & \text{otherwise}, \end{cases} \tag{30}$$

$$d_{\text{avail}} = s - r_{\text{apex}}, \tag{31}$$

$$\rho_{C6} = \max(0,\, d_{\text{relief}} - d_{\text{avail}}). \tag{32}$$

Constraint $C6$ is satisfied iff

$$\rho_{C6} \le \varepsilon,$$

which corresponds to the intended inequality

$$T\tan(\gamma/2) \le s - r_{\text{apex}}.$$

References: This is an AKDE validity proxy derived from the minor-cut clearance concept, not a closed-form inequality from Liu et al. Conceptual source: Liu et al., major/minor-cut discussion around Figure 2. Implementation authority: `kirigami/model/constraints.ts`.

6 Rendered Net Geometry

The exported SVG pattern uses the same derived geometry plus the optional perimeter input L_o.

6.1 Outer polygon span

The implementation first chooses

$$L_o^* = \begin{cases} L_o, & L_o > 0, \\ L, & \text{otherwise}. \end{cases}$$

Then it computes

$$\alpha_{\max} = \frac{\tau}{2} - 10^{-3}, \tag{33}$$

$$\alpha_o = \min\left(\max\left(\arcsin\left(\min\left(1, \frac{L_o^*}{2s}\right)\right), 10^{-4}\right), \alpha_{\max}\right). \tag{34}$$

References: This is an AKDE rendering construction for the outer perimeter chord span. The triangle relation behind $\arcsin(L_o^*/(2s))$ comes from the same law-of-sines / chord geometry used above; see OpenStax, "10.1 Non-right Triangles: Law of Sines." Implementation authority: `kirigami/model/pattern.ts`.

Here α_o is the half-span of each polygon outer base on the radius-s circle.

6.2 Rendered molecule outer chord

Because the perimeter alternates polygon and molecule spans, the rendered molecule chord becomes

$$w_{\text{rendered}} = 2s\sin\left(\max\left(0, \frac{\tau}{2} - \alpha_o\right)\right). \tag{35}$$

References: AKDE rendering-specific chord formula from `kirigami/model/pattern.ts`; it follows from circle-chord geometry after allocating perimeter span α_o to the polygon outer base.

6.3 Minor cut passed to rendering

The renderer uses

$$\ell_{\text{render}} = \text{computeMinorCutLength}(\gamma,\, w_{\text{rendered}},\, T,\, \theta,\, r_{\text{apex}})\,. \tag{36}$$

With the active `fold-reach` branch, this reduces to

$$\ell_{\text{render}} = \sqrt{\left(\frac{w_{\text{rendered}}}{2}\right)^2 + r_{\text{apex}}^2}.$$

References: AKDE rendering specialization of the active `fold-reach` branch from `kirigami/model/geometry.ts` and `kirigami/model/pattern.ts`.

7 Default Input Case

The default state is

$$N = 6, \qquad L = 100 \text{ mm}, \qquad L_o = 100 \text{ mm}, \qquad T = 1 \text{ mm},$$

with

$$R = \frac{L}{2\sin(\pi/N)}, \qquad H = R.$$